

# DBMS Fundamentals & Architecture

---

## 1. What is a DBMS?

---

**A DBMS (Database Management System) is a collection of interrelated data, plus a set of programs to access that data.**

- **Data** — raw, unprocessed facts.
  - **Information** — processed data (data that's been given meaning/context).
  - **Metadata** — data *about* data (e.g., a file's author, creation date, size — or, in a DB context, a table's schema, column types, constraints).
- 

## 2. File System vs DBMS — why DBMS exists

---

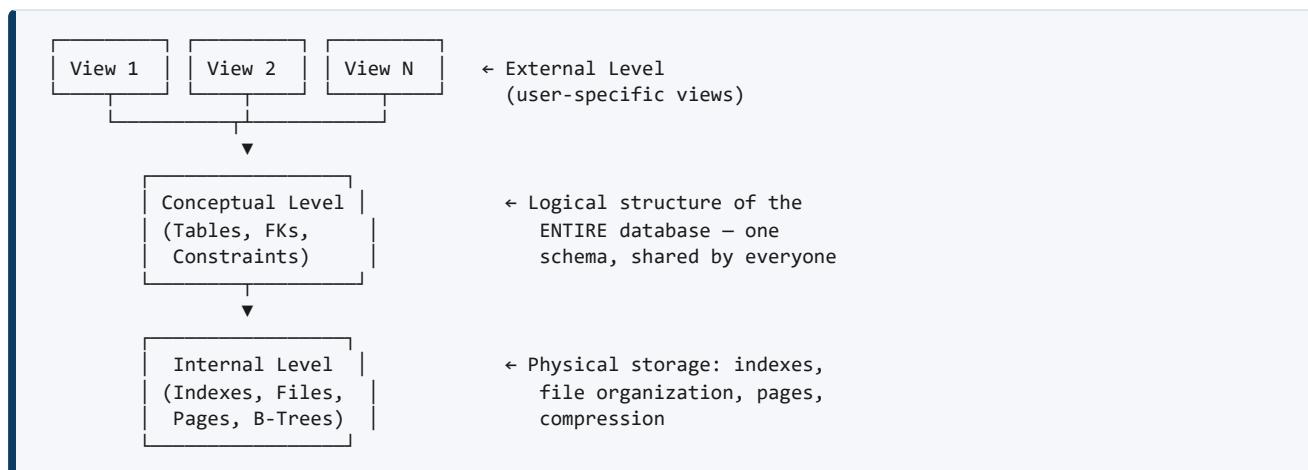
Before DBMSs, applications managed their own data directly as files. This created systemic problems that a DBMS was built specifically to solve:

| Problem in file systems                                                                                                  | How a DBMS solves it                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data redundancy &amp; inconsistency</b> — the same data duplicated across multiple files, which can drift out of sync | Centralized storage with controlled redundancy via normalization and relational structure                                                                              |
| <b>Difficulty accessing data</b> — every new access pattern needs new application code                                   | A query language (SQL) lets you ask arbitrary questions without writing new programs                                                                                   |
| <b>Data isolation</b> — related data scattered across incompatible file formats                                          | Unified schema — related data lives together, in one consistent structure                                                                                              |
| <b>Integrity problems</b> — no built-in way to enforce rules (e.g., "ID must not be empty")                              | Built-in integrity constraints (primary key, foreign key, domain constraints, etc.)                                                                                    |
| <b>Atomicity problems</b> — a multi-step operation (e.g., a fund transfer) can fail halfway, leaving inconsistent data   | Transactions — guarantee <b>all-or-nothing</b> execution (e.g., a transfer either fully completes or is fully rolled back — money is never "debited but not credited") |
| <b>Concurrent access anomalies</b> — multiple programs modifying the same file simultaneously can corrupt it             | Concurrency control subsystem — coordinates simultaneous access without conflicts (see the Concurrency Control topic)                                                  |
| <b>Security problems</b> — a file system's access control is coarse (whole file, not specific fields)                    | Authentication + authorization — different users can be given different privileges, down to the level of a specific table, row, or column                              |

**Interview soundbite:** "Every one of DBMS's core subsystems exists because of a specific, concrete failure mode of raw file-based data management — redundancy, isolation, integrity, atomicity, concurrency, and security. A DBMS isn't just 'a fancier way to store files' — it's a direct response to a documented list of things that reliably go wrong without it."

### 3. The Three-Schema Architecture (ANSI/SPARC) — Three Levels of Abstraction

**Purpose:** separate *what data means* from *how it's stored* from *how a specific user sees it* — enabling data independence (§4) and hiding complexity from each audience.



| Level             | Also called    | Describes                                                                                      | Who uses it                     |
|-------------------|----------------|------------------------------------------------------------------------------------------------|---------------------------------|
| <b>External</b>   | View level     | User/application-specific subsets of the data                                                  | End users, application programs |
| <b>Conceptual</b> | Logical level  | The complete logical structure — entities, relationships, constraints — independent of storage | DB designers, developers        |
| <b>Internal</b>   | Physical level | How data is actually stored — indexes, file organization, compression                          | DBMS itself / DBAs              |

### Worked example (a bank):

- **External:** a bank clerk's view shows only `CustomerName, Balance` ; a manager's view shows the complete account record.
- **Conceptual:** the full logical schema — `Customer(CustID, Name, Address)` , `Account(AccNo, Type, Balance)` , with the relationship "Customer owns Account."
- **Internal:** the actual on-disk representation — e.g., the `Account` table's rows stored in pages, indexed via a B+-tree on `AccNo` (see the Indexing topic).

**Interview soundbite:** "The three-schema architecture is really about need-to-know. A user needs to know only their view. An application developer needs to know the logical schema, not the physical layout. Only the DBMS (and DBAs) need to know the physical details. Each level hides exactly the complexity the layer above it doesn't need to care about."

## 4. Data Independence

**Data independence** = the ability to change the schema at one level **without** needing to change the schema (or application code) at the level above it.

| Type                              | Change allowed                                                                                                             | What must remain unaffected                                                                          |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Logical Data Independence</b>  | Changing the <b>conceptual</b> schema (e.g., adding a column, splitting/merging tables)                                    | The <b>external</b> level — existing applications/views that don't use the changed part keep working |
| <b>Physical Data Independence</b> | Changing the <b>internal</b> schema (e.g., switching storage devices, adding/removing indexes, changing file organization) | The <b>conceptual</b> level — the logical structure and every application built on it                |

### Worked examples:

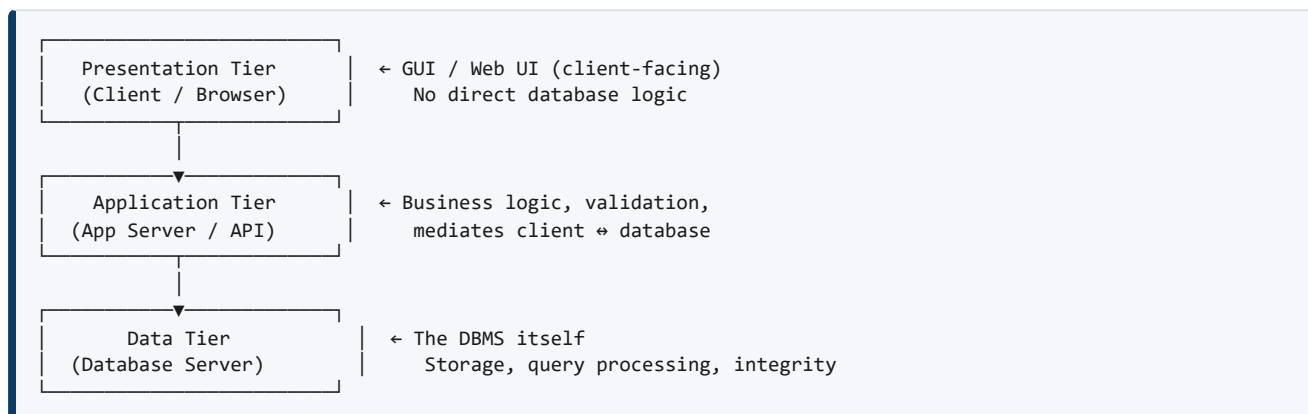
- *Logical independence*: adding an `email` column to `Customer` — existing applications that don't reference `email` keep working unmodified.
- *Physical independence*: migrating from HDD to SSD, or reorganizing a table's on-disk blocks for performance — SQL queries against that table are completely unaffected.

**Which is "bigger"?** Physical data independence is generally considered *easier* to achieve (storage details are naturally hidden below the conceptual level already) — logical data independence is *harder*, because application code can accidentally become tightly coupled to the exact logical structure (e.g., `SELECT *` breaking if columns are reordered), which is exactly why disciplined schema design and views matter.

**Interview soundbite:** "Data independence is not about hiding data — it's about hiding *change*. The whole point is that a change at one level shouldn't ripple upward and force rewrites everywhere above it."

## 5. Three-Tier Architecture

A **client-server** architecture that separates an application into three distinct layers — note this is a different (though related) concept from the three *schema* levels above; this is about *where application logic physically runs*, not about database schema abstraction.



| Tier                | Responsibility                                         | Example technologies                |
|---------------------|--------------------------------------------------------|-------------------------------------|
| <b>Presentation</b> | User interaction only — displays data, takes input     | Browsers, mobile/desktop apps       |
| <b>Application</b>  | Business logic, validation, mediates client ↔ database | Flask, Django, Express, API servers |
| <b>Data</b>         | Storage, query processing, integrity, backups          | Oracle, MySQL, PostgreSQL, MongoDB  |

## Why separate into three tiers rather than two (client talks directly to DB)?

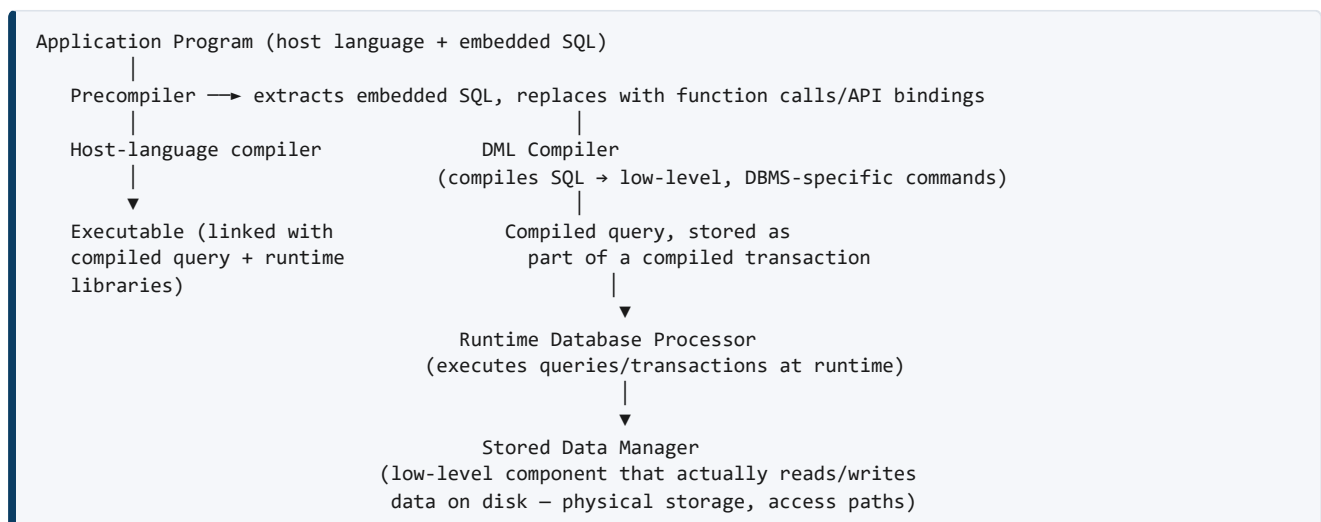
- **Separation of concerns** — the presentation layer doesn't need to know SQL; the database doesn't need to know about UI.
- **Independent scaling** — the application tier (often the bottleneck under load) can be scaled out independently of the database.
- **Security** — the database is never directly exposed to the client; all access is mediated and validated by the application tier.
- **Independent development/updates** — each tier has its own infrastructure and can be developed, deployed, and updated somewhat independently.

**Interview soundbite:** "Three-tier architecture is the same 'separate concerns into layers' principle you see in OS design and network protocol stacks, applied to application architecture — the client shouldn't need to know SQL, and the database shouldn't need to know anything about how data is displayed."

## 6. Database Users

| User type                           | Technical skill                    | What they do                                                                                                                                                         |
|-------------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DBA (Database Administrator)</b> | High                               | Manages the entire database system — schema design, permissions, performance tuning, backups                                                                         |
| <b>Application Programmers</b>      | High (general-purpose programming) | Write general-purpose programs (embedding SQL, e.g., via a host language like C++/Python) that interact with the database                                            |
| <b>Casual users</b>                 | Some technical skill               | Write ad-hoc SQL queries directly for one-time/irregular data analysis needs                                                                                         |
| <b>Parametric (naive) users</b>     | Low                                | Use predefined, pre-built application interfaces (forms, menus) for repetitive tasks — never write SQL themselves (e.g., a bank teller using a fixed deposit screen) |

## 7. Query Processing Architecture — how a query actually gets executed



- **DDL compiler** — processes schema definitions ( `CREATE TABLE` , etc.) and stores the result in the **system catalog** (the DBMS's metadata repository — this is literally where the conceptual schema lives).
- **Query compiler** (includes the **optimizer**) — converts a SQL query into an efficient internal execution plan.

- **Precompiler** — for embedded SQL (SQL written inside a general-purpose host language like C++), extracts the SQL statements and replaces them with host-language function calls/API bindings, before the surrounding code goes through a normal host-language compiler.
- **DML compiler** — compiles the extracted SQL into low-level, DBMS-specific instructions.
- **Runtime database processor** — actually executes queries/transactions at runtime — this is the component doing real work each time a query runs, as opposed to the compilers, which do their work once, ahead of time.
- **Stored data manager** — the lowest-level component; physically reads and writes data to disk, manages access paths (indexes, file organization).

**Interview soundbite:** "This pipeline mirrors general compiler architecture — parse, compile/optimize, then execute — applied specifically to queries. The system catalog is the crucial piece tying it together: it's the DBMS's own metadata database, storing the schema information every other component needs to correctly interpret and execute queries against actual data."

---

## Interview Questions With Answers

---

### Q1. List three concrete problems with raw file-based data management, and how DBMS-specific mechanisms solve each.

**Answer:** (1) Data redundancy/inconsistency — files duplicate data across independent copies that can drift apart; a DBMS uses a centralized schema with normalization to minimize controlled redundancy. (2) Atomicity problems — a multi-step file update can fail partway, leaving inconsistent data (e.g., money debited but not credited); a DBMS's transaction mechanism guarantees all-or-nothing execution. (3) Concurrent access anomalies — multiple programs writing to the same file simultaneously can corrupt it; a DBMS's concurrency control subsystem coordinates simultaneous access to prevent this.

### Q2. Explain the three-schema architecture and why each level exists.

**Answer:** The external level provides user/application-specific views, hiding irrelevant data and providing security by restricting what each user can see. The conceptual level defines the single, complete logical structure of the entire database (entities, relationships, constraints), independent of physical storage — this is what application developers actually design against. The internal level defines the actual physical storage (indexes, file organization, compression), hidden from everyone except the DBMS and DBAs. Each level exists to hide the complexity/details irrelevant to the layer above it, while enabling data independence between levels.

### Q3. What's the difference between logical and physical data independence? Give an example of each.

**Answer:** Logical data independence is the ability to change the conceptual schema (e.g., add a column, split a table) without needing to change the external level — existing applications/views not using the changed part keep working. Physical data independence is the ability to change the internal (physical) storage structure (e.g., switch storage devices, add an index, change file organization) without needing to change the conceptual schema at all. Example of logical: adding an `email` column to `Customer` doesn't break existing queries that don't reference it. Example of physical: migrating a table from HDD to SSD, or adding a new index, doesn't require rewriting any SQL queries against that table.

### Q4. Why is logical data independence generally considered harder to fully achieve than physical data independence?

**Answer:** Physical storage details are already naturally isolated below the conceptual level — applications never interact with disk layout directly in the first place, so changing it has a clean boundary to respect. Logical data independence is harder because application code can inadvertently become tightly coupled to the exact current logical structure — for example, a query using `SELECT *` will break if columns are reordered or a column is removed, even though that's exactly the kind of conceptual-schema change logical independence is supposed to shield applications from; achieving true logical independence requires disciplined practices (like using views, and avoiding overly structure-dependent queries) rather than being automatic.

### Q5. In three-tier architecture, why shouldn't the presentation tier talk directly to the data tier?

**Answer:** Direct client-to-database communication would require exposing database credentials/access directly to the client, and would force every client to embed database-specific logic (SQL, connection handling) and business rules, duplicating that logic across every client type (web, mobile, etc.) and creating a serious security exposure. The application tier exists specifically to centralize business logic and validation once, mediate and control what the client can actually do to the data, and keep the database itself never directly reachable from untrusted client code.

### Q6. What is the system catalog, and why is it central to query processing?

**Answer:** The system catalog is the DBMS's own metadata repository — it stores the schema information (table definitions, column types, constraints, indexes) produced by the DDL compiler when schemas are defined. It's central to query processing because every other component (the query compiler/optimizer, the DML compiler, the runtime processor) needs this metadata to correctly interpret what a query is actually referring to and how to execute it efficiently — without the catalog, there would be no authoritative source of "what does this table/column actually look like" for any component to consult.

### **Q7. What's the difference between a casual user and a parametric user of a database?**

**Answer:** A casual user has some technical skill and writes ad-hoc SQL queries directly, typically for one-time or irregular data analysis needs (e.g., a data analyst running a custom query). A parametric (naive) user has little to no technical/SQL skill and only interacts with the database through predefined, pre-built application interfaces — forms, menus, fixed screens — designed for repetitive, well-understood tasks (e.g., a bank teller processing a standard deposit through an ATM-like interface), never writing or seeing SQL at all.

### **Q8. Scenario: A company wants to move its database from an on-premise server to a cloud-hosted instance, changing the underlying storage hardware and file organization entirely, without touching any application code. Which DBMS property makes this possible, and at which schema level does the change occur?**

**Answer:** This is enabled by **physical data independence** — the change is entirely at the **internal (physical) level** (different storage hardware, potentially different file organization/indexing strategy on the new platform), while the **conceptual level** (the logical schema — tables, columns, relationships, constraints) stays exactly the same. Because application code is written against the conceptual schema, not the physical storage details, and physical data independence guarantees the conceptual level is shielded from internal-level changes, the application code requires no modification at all — only the DBA-level configuration of the new physical storage needs to change.